

Architecture Overview

Components

Component	Role
React 18 + TypeScript frontend	URL creation form, link management, analytics dashboard. Served via Nginx in production, Vite dev server locally.
Spring Boot 3.2.5 API (Java 17)	REST API for URL CRUD, redirects, analytics. Owns all business logic.
PostgreSQL	System of record for <code>short_urls</code> and <code>click_events</code> . The <code>UNIQUE</code> constraints on <code>code</code> and <code>custom_alias</code> are the actual source of truth for uniqueness.
Redis	Two distinct jobs: atomic Base62 counter for code generation (INCR/batch allocation), and a cache-aside store for the redirect hot path.
Kafka	Decouples click recording from the redirect response. 3-partition <code>click-events</code> topic, keyed by <code>short_code</code> .
Claude API (optional)	Runtime safety classifier on the write path. This isn't a coding tool here, it's a functional service dependency the app calls at request time. Fails open if unconfigured or unreachable.

Control Flow

Write path: POST /api/v1/urls

```
Browser -> React -> Spring Boot
      -> Rate Limiter (Bucket4j, 10 req/min per IP)
      -> AI Safety Check (Claude API): unsafe -> 422 + reason
      -> Redis Counter (INCRBY batch of 1000 -> Base62) -> PostgreSQL (short_urls)
```

Every stage before the DB write can reject the request (429, 422, 400, 409). Nothing downstream of a rejection runs.

Read path: GET /{code}

```
Browser -> Spring Boot -> Redis cache
      hit -> 302 immediately
      miss -> PostgreSQL fallback -> repopulate cache
      -> (always, async) -> Kafka (click-events) -> Consumer -> PostgreSQL (click_events)
```

The redirect response never waits on the Kafka publish or the consumer. That's deliberate: analytics is allowed to be eventually consistent, but redirects are not.

Key Decisions

302, not 301, for redirects. 301 would let browsers cache the redirect permanently, which breaks click analytics after the first visit per browser and makes link deletion/expiry impossible to enforce. Full trade-off is in `RISKS_AND_TRADEOFFS.md`.

Counter-based Base62 codes, not hashing. There's zero collision risk with a counter versus a non-zero collision probability with truncated MD5/SHA. Redis INCR is atomic, so this works fine under concurrent writers without any locking. Batched allocation (1000 IDs per Redis round trip) trades a bounded number of wasted IDs on restart for far fewer Redis calls, which felt like the right trade at this scale.

Kafka sits on the read path, not the write path. Click events are the highest-volume and most disposable data in the system, so losing a few during a Kafka outage is acceptable. A slow or failed redirect is not. Producer failures are caught and logged, never propagated up.

Service layer split by read/write, not by entity. WriteService, ReadService, AnalyticsService, UrlInfoService. The read path is hot and latency-sensitive; the write path is cold and can afford extra checks. Those are genuinely different performance and testing needs, so I kept them as separate classes instead of one service per JPA entity.

AI safety check fails open. If ANTHROPIC_API_KEY is unset or the API call errors or times out, the URL is allowed through. Availability of the core shortening function matters more than completeness of the safety layer for this prototype. That's a documented scope decision, not an oversight.

Execution Approach (AI-Assisted Engineering)

Implementation was done with Claude Code in an IDE/CLI-driven workflow: class-by-class specification-first prompting, engineer review of every generated file, and an explicit accept/edit/reject decision recorded for each unit of generated code. AI_WORKFLOW.md has the full traceability record, including how this architecture doc was translated into structured, AI-consumable implementation specs.

Data Model

short_urls: id, code (UNIQUE), custom_alias (UNIQUE, nullable), original_url, created_at, expires_at (nullable), created_by_ip.

click_events: id, short_code (indexed), clicked_at, ip_address, referrer, user_agent.

Both tables are created via Hibernate ddl-auto: update in this prototype. See RISKS_AND_TRADEOFFS.md for why that's fine here but not a production recommendation.

API Surface

Method	Path	Purpose
POST	/api/v1/urls	Create a short URL (optional custom alias, optional expiry)
GET	/api/v1/urls	List all URLs
GET	/api/v1/urls/{code}	URL metadata
DELETE	/api/v1/urls/{code}	Delete a URL
GET	/ {code}	302 redirect. Root-level, not under /api/v1, so short links stay short.
GET	/api/v1/analytics/{code}	Total clicks, clicks by day, top referrers
GET	/actuator/health	Liveness/readiness for orchestration
GET	/swagger-ui.html	Interactive API docs (springdoc-openapi)

Deployment Topology (prototype)

Single Docker Compose stack: postgres, redis, zookeeper, kafka, app (Spring Boot), frontend (Nginx). No orchestration, no multi-instance coordination beyond what Redis and Kafka provide natively. Production topology and the gaps that come with this simpler setup are covered in FINAL_ENGINEERING_SUMMARY.md.